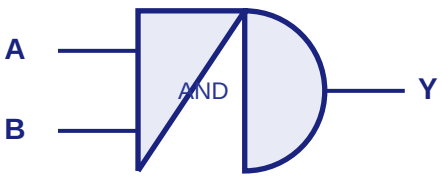
RTL to GDSII

Complete ASIC Implementation Guide

Design Under Test: 2-Input AND Gate

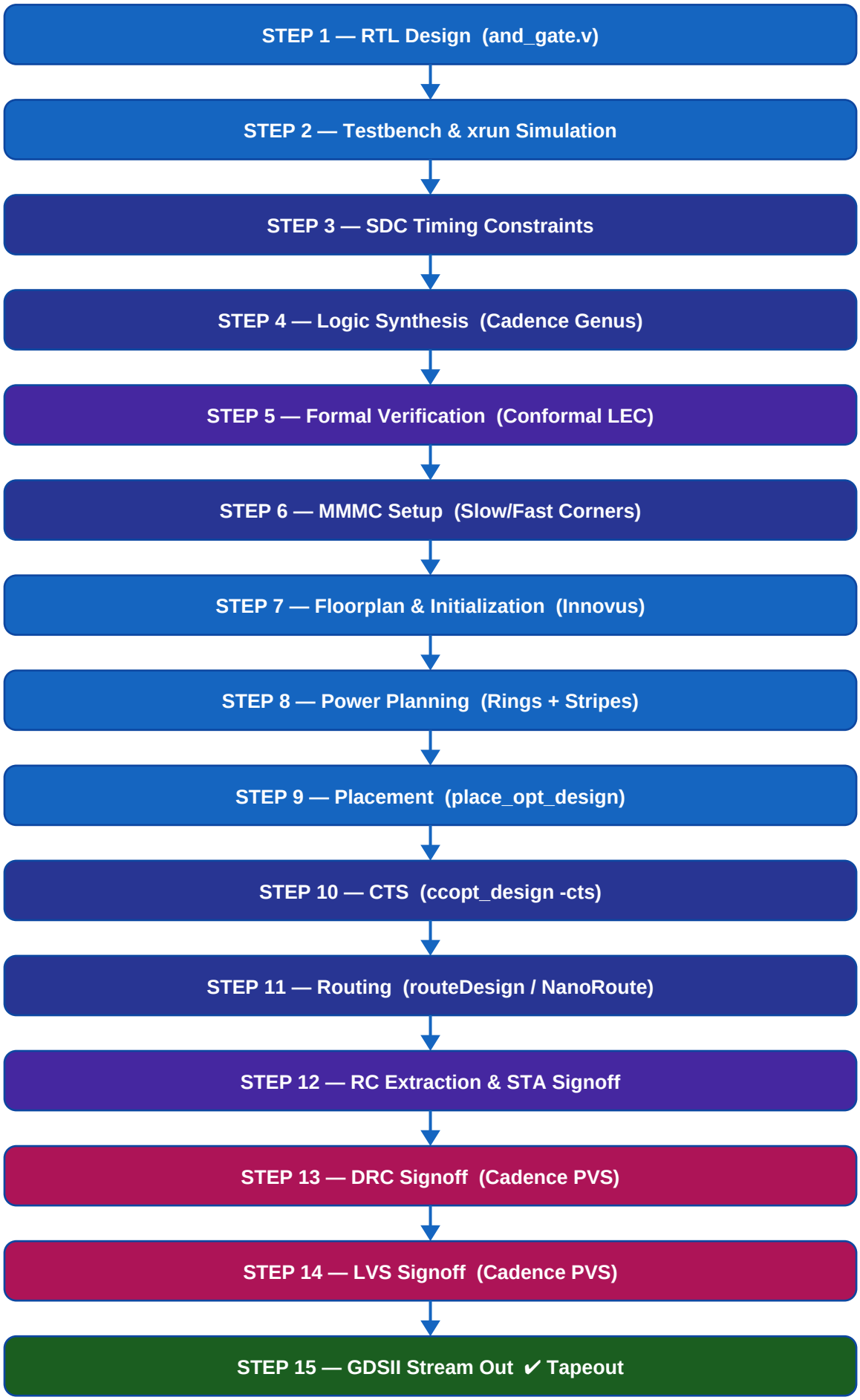


A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Technology Node	45 nm GPDK (Generic PDK)
EDA Tools	Cadence Genus · Innovus · Conformal LEC · PVS
Simulator	Cadence Xcelium (xrun)
Design Language	Verilog-2001
Target Frequency	100 MHz (10 ns period, combinational)
Output Deliverables	Gate netlist · GDSII · SPEF · SDF · DRC/LVS reports

This guide walks through every phase of digital ASIC implementation—from writing synthesizable RTL Verilog, running functional simulation with Cadence Xcelium (xrun), logic synthesis with Genus, formal equivalence checking with Conformal LEC, full Place & Route with Innovus, through physical verification (DRC & LVS) with PVS and final GDSII tapeout.

RTL-to-GDSII Implementation Flow



STEP 1 — RTL Design: 2-Input AND Gate

The AND gate is the fundamental combinational logic building block. Its output Y is HIGH only when **both** inputs A and B are HIGH. The synthesizable Verilog RTL below uses a single continuous assignment which Genus maps directly to an AND2 standard cell.

Truth Table

Input A	Input B	Output Y = A & B
0	0	0
0	1	0
1	0	0
1	1	1

Verilog RTL Source — and_gate.v

■ src/and_gate.v

```
// File: src/and_gate.v
// Module: and_gate
// Desc  : 2-input AND gate — synthesizable Verilog-2001
//
// Port Map:
//   a  — Input  A
//   b  — Input  B
//   y  — Output Y = A & B
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

module and_gate (
    input  wire a,   // Input A
    input  wire b,   // Input B
    output wire y    // Output Y = A AND B
);

    // Continuous assignment — synthesizes to a single AND2 standard cell
    assign y = a & b;

endmodule
```

Key RTL Coding Guidelines

- **assign y = a & b;** — one-line combinational logic, no always block needed.
- Use **input wire** / **output wire** for combinational ports (no reg on output).
- No clock, no reset — pure combinational design has zero sequential elements.
- Bitwise **&** operator synthesizes to an AND gate; logical **&&** is for conditions.
- The module compiles under **Verilog-2001** (ANSI port style).

STEP 2 — Testbench & Cadence Xcelium Simulation

Functional simulation verifies that the RTL behaves correctly before synthesis. Cadence **Xcelium** is the industry-standard Verilog/SystemVerilog simulator invoked through the **xrun** front-end command. The testbench applies all four input combinations of a 2-input AND gate and self-checks the output.

Testbench Source — and_gate_tb.v

tb/and_gate_tb.v

```
`timescale 1ns/1ps

module and_gate_tb;

    // ■■ DUT signals ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    reg  a, b;
    wire y;

    // ■■ DUT instantiation ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    and_gate uut (.a(a), .b(b), .y(y));

    // ■■ Check task ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    integer pass_cnt = 0, fail_cnt = 0;

    task check;
        input exp;
        input [63:0] label;
        begin
            #1;
            if (y === exp) begin
                pass_cnt = pass_cnt + 1;
                $display("[PASS] %0s | a=%b b=%b y=%b", label, a, b, y);
            end else begin
                fail_cnt = fail_cnt + 1;
                $display("[FAIL] %0s | a=%b b=%b y=%b (exp %b)",
                          label, a, b, y, exp);
            end
        end
    endtask

    // ■■ Stimulus: all 4 truth table combinations ■■■■■■■■■■■■■■■■■■
    initial begin
        $display("=== AND Gate Testbench ===");

        a=0; b=0; check(1'b0, "0 AND 0");   // Y must be 0
        a=0; b=1; check(1'b0, "0 AND 1");   // Y must be 0
        a=1; b=0; check(1'b0, "1 AND 0");   // Y must be 0
        a=1; b=1; check(1'b1, "1 AND 1");   // Y must be 1

        $display("Results: %0d PASSED, %0d FAILED", pass_cnt, fail_cnt);
        if (fail_cnt == 0) $display(">>> ALL TESTS PASSED <<<");
        $finish;
    end

endmodule
```

Cadence Xcelium (xrun) Commands

■ RTL Simulation

```
xrun src/and_gate.v tb/and_gate_tb.v \
    -timescale 1ns/1ps \
    -access +rwc \
    -log sim_rtl.log
```

■ RTL Sim + Waveform (SHM)

```
xrun src/and_gate.v tb/and_gate_tb.v \
    -timescale 1ns/1ps \
    -access +rwc \
    -define DUMP_SHM \
    -shm \
    -log sim_rtl_waves.log
```

■ Gate-Level Sim (post-synthesis)

```
xrun netlist/and_gate_synth.v \
    $GPDK045/verilog/fast_vdd1v0_basicCells.v \
    tb/and_gate_tb.v \
    -timescale 1ns/1ps \
    -access +rwc \
    -delay_mode zero \
    +notimingcheck \
    -log sim_gate.log
```

■ Post-Route Sim with SDF backannotation

```
xrun netlist/and_gate_postroute.v \
    $GPDK045/verilog/fast_vdd1v0_basicCells.v \
    tb/and_gate_tb.v \
    -timescale 1ns/1ps \
    -access +rwc \
    -sdf_cmd_file scripts/sdf_annotate.cmd \
    -log sim_postroute.log
```

■ Batch file approach (xrun -f)

```
xrun -f scripts/sim_rtl.f
```

Simulation File List — scripts/sim_rtl.f

■ scripts/sim_rtl.f

```
// Cadence xrun file-list for AND gate RTL simulation
// Usage: xrun -f scripts/sim_rtl.f

src/and_gate.v          // RTL source
tb/and_gate_tb.v        // Self-checking testbench

-timescale 1ns/1ps
-access +rwc
-64bit
-clean
-log sim_rtl.log
-run
```

Expected xrun Console Output

Expected Console Output

```
xrun: 20.09-s005
Loading  src/and_gate.v
Loading  tb/and_gate_tb.v
Elaborating...

=== AND Gate Testbench ===
[PASS] 0 AND 0 | a=0 b=0 y=0
[PASS] 0 AND 1 | a=0 b=1 y=0
[PASS] 1 AND 0 | a=1 b=0 y=0
[PASS] 1 AND 1 | a=1 b=1 y=1
Results: 4 PASSED, 0 FAILED
>>> ALL TESTS PASSED <<<

Simulation complete.
```

STEP 3 — SDC Timing Constraints

The Synopsys Design Constraints (SDC) file provides the timing specification used by both Genus (synthesis) and Innovus (PnR). Since the AND gate is purely combinational, a virtual clock is defined as a timing reference for I/O budgets.

constraints/and_gate.sdc

```
##################################################################
# File: constraints/and_gate.sdc
# Design: and_gate (combinational)
# Tool:   Cadence Genus / Innovus
##################################################################

# Virtual clock — 100 MHz reference for I/O timing budget
create_clock -name vclk -period 10.0

# Input arrival times (max = setup, min = hold budget)
set_input_delay  -max 2.0 -clock vclk [get_ports {a b}]
set_input_delay  -min 0.5 -clock vclk [get_ports {a b}]

# Output required times
set_output_delay -max 2.0 -clock vclk [get_ports y]
set_output_delay -min 0.5 -clock vclk [get_ports y]

# Drive strength of input ports (driven by INVX1 strength driver)
set_driving_cell -lib_cell INVX1 [get_ports {a b}]

# Capacitive load on output port (20 fF)
set_load -pin_load 0.020 [get_ports y]
```

STEP 4 — Logic Synthesis with Cadence Genus

Cadence **Genus** translates the synthesizable Verilog RTL into a technology-specific gate-level netlist using the GPDK045 standard cell library. Synthesis is performed in three sequential stages: Generic → Technology Map → Optimization.

Run Command

```
genus -f scripts/genus_synth.tcl
```

Full Synthesis Script — scripts/genus_synth.tcl

■ scripts/genus_synth.tcl

```
################################################################
# Cadence Genus Synthesis Script: and_gate
# Usage: genus -f scripts/genus_synth.tcl
################################################################

# ■■ 0. Create output directories ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
file mkdir reports
file mkdir netlist

# ■■ 1. Library search paths ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
set GPDK /home/ec/install/FOUNDRY/lan/flow/t1u1/reference_libs
set LIB  /home/ec/install/FOUNDRY/digital/45nm/dig/lib

set_db / .init_lib_search_path [list \
    $LIB \
    $GPDK/GPDK045/gsclib045_svt_v4.4/gsclib045/timing ]

set_db / .init_hdl_search_path {./src}

# ■■ 2. Target technology library (.lib) ■■■■■■■■■■■■■■■■■■■■■■■■
set_db / .library { $LIB/typical.lib }

# ■■ 3. Physical LEF (for area/wireload estimation) ■■■■■■■■■■■■■
set_db / .lef_library [list \
    $GPDK/GPDK045/gsclib045_svt_v4.4/gsclib045_tech/lef/gsclib045_tech.lef \
    $GPDK/GPDK045/gsclib045_svt_v4.4/gsclib045/lef/gsclib045_macro.lef ]

# ■■ 4. Read HDL & elaborate ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
read_hdl -verilog src/and_gate.v
elaborate and_gate

check_design -unresolved

# ■■ 5. Read timing constraints ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
read_sdc constraints/and_gate.sdc
report_clocks > reports/clocks_synth.rpt

# ■■ 6. Three-stage synthesis ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Stage A: RTL → GTECH (technology-independent logic)
syn_generic
report_timing > reports/timing_generic.rpt
```

```
# Stage B: GTECH → 45 nm standard cell gates
syn_map
report_timing > reports/timing_mapped.rpt

# Stage C: Timing/area/power optimization
syn_opt

# ■■ 7. QoR reports ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
check_design -all         > reports/check_design.rpt
report_timing -nworst 10  > reports/timing_synth.rpt
report_area               > reports/area_synth.rpt
report_power              > reports/power_synth.rpt
report_gates              > reports/gates_synth.rpt
report_qor                > reports/qor_synth.rpt

# ■■ 8. Export gate-level netlist & SDC ■■■■■■■■■■■■■■■■■■■■■■■■
write_hdl  > netlist/and_gate_synth.v
write_sdc  > netlist/and_gate_synth.sdc

puts "=== Genus Synthesis Complete ==="
```

Key Genus Commands Reference

Command	Description
`genus -f <script>`	Launch Genus and execute the Tcl script
`set_db / .library {...}`	Set target .lib technology library
`set_db / .lef_library {}`	Set physical LEF files for geometry info
`read_hdl -verilog <file>`	Read Verilog RTL source file
`elaborate <top>`	Compile, resolve hierarchy, check design
`check_design -unresolved`	Flag any blackbox / unresolved references
`read_sdc <file>`	Import timing constraints (SDC format)
`syn_generic`	Stage A: RTL to GTECH (tech-independent)
`syn_map`	Stage B: Map GTECH to target cell library
`syn_opt`	Stage C: Timing/area/power optimization
`report_timing -nworst N`	Show N worst setup timing paths
`report_area`	Report cell count, area, density
`report_power`	Report dynamic + static power
`write_hdl`	Export gate-level netlist (Verilog)
`write_sdc`	Export synthesized timing constraints

Expected Synthesized Gate Netlist — netlist/and_gate_synth.v

■ netlist/and_gate_synth.v (Genus output)

```
// Genus auto-generated gate-level netlist
// Technology: 45nm GPDK (SVT)

module and_gate ( a, b, y );
    input  a, b;
    output y;
    // Genus maps 'assign y = a & b' to a single AND2X1 standard cell
```

```
    AND2X1 U_AND ( .A(a), .B(b), .Y(y) );

endmodule
```

STEP 5 — Formal Verification with Conformal LEC

Cadence **Conformal LEC** formally proves (mathematically, without simulation) that the synthesized gate-level netlist is *logically equivalent* to the original RTL. For a simple AND gate this is trivially provable but the same methodology scales to million-gate designs.

Run Command

```
lec -xl -nogui -do scripts/conformal_lec.tcl
```

LEC Script — scripts/conformal_lec.tcl

■ scripts/conformal_lec.tcl

```
##################################################################
# Cadence Conformal LEC: RTL vs. Gate Netlist
# Usage: lec -xl -nogui -do scripts/conformal_lec.tcl
##################################################################

set_log_file lec_results.log -replace

# Flattening options
set_flatten_model -seq_constant -all
set_flatten_model -nodeset_to_constraint

# ■■ 1. Golden design = RTL ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
read_library -liberty /home/ec/install/FOUNDRY/digital/45nm/dig/lib/typical.lib -both
read_design src/and_gate.v -verilog -golden -lastmod
set_root_module and_gate -golden

# ■■ 2. Revised design = synthesized gate netlist ■■■■■■■■■■■■■■■■
read_design netlist/and_gate_synth.v -verilog -revised -lastmod
set_root_module and_gate -revised

# ■■ 3. Map compare points ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
set_system_mode lec
map_key_points
report_unmapped_points

# ■■ 4. Run formal comparison ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
add_compare_points -all
compare

# ■■ 5. Report results ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
report_compare_data -nonequivalent
puts "=== LEC: Formal proof complete ==="
```

Expected LEC Output (PASS)

■ Expected Console Output

```
Reading library: typical.lib
Reading golden  : src/and_gate.v
Reading revised : netlist/and_gate_synth.v
Mapping key points...
```

```
Mapped output  :  y  <-->  y

Running formal comparison...

Compared points  :  1
Equivalent       :  1
Non-equivalent   :  0
Abort            :  0

>>> FORMAL VERIFICATION PASSED <<<
```

STEP 6 — Multi-Mode Multi-Corner (MMMC) Setup

MMMC analysis runs setup and hold checks simultaneously across the worst-case (Slow, 125°C) and best-case (Fast, 0°C) process-voltage-temperature corners.

scripts/mmmc.tcl

```
##################################################################
# File: scripts/mmmc.tcl — Cadence Innovus MMMC configuration
##################################################################

set GPDK /home/ec/install/FOUNDRY/lan/flow/t1u1/reference_libs
set TIM  $GPDK/GPDK045/gsclib045_svt_v4.4/gsclib045/timing

# Slow corner: Worst-case setup (slow process, 125°C, low VDD)
create_library_set -name slow_libs -timing [
    list $TIM/slow_vdd1v0_basicCells.lib \
         $TIM/slow_vdd1v0_multibitsDFF.lib]

# Fast corner: Best-case hold (fast process, 0°C, high VDD)
create_library_set -name fast_libs -timing [
    list $TIM/fast_vdd1v0_basicCells.lib \
         $TIM/fast_vdd1v0_multibitsDFF.lib]

# RC corners (interconnect resistance/capacitance model)
create_rc_corner -name slow_rc -temperature 125
create_rc_corner -name fast_rc -temperature 0

# Delay corners (cell timing + interconnect RC)
create_delay_corner -name slow_delay \
    -library_set slow_libs -rc_corner slow_rc
create_delay_corner -name fast_delay \
    -library_set fast_libs -rc_corner fast_rc

# Constraint mode (SDC exported from Genus)
create_constraint_mode -name func_mode \
    -sdc_files {netlist/and_gate_synth.sdc}

# Analysis views
create_analysis_view -name slow_view \
    -constraint_mode func_mode -delay_corner slow_delay
create_analysis_view -name fast_view \
    -constraint_mode func_mode -delay_corner fast_delay

# Assign views: Setup = slow, Hold = fast
set_analysis_view -setup {slow_view} -hold {fast_view}
```

STEP 7–11 — Place & Route with Cadence Innovus

Cadence **Innovus** takes the gate-level netlist and physically places standard cells on a silicon die, connects power networks, synthesizes the clock tree (for sequential designs), routes all signal wires, and verifies the physical design before producing the final GDSII layout.

Master Run Command

Innovus Invocation

```
# Run complete PnR flow in batch
innovus -stylus -f scripts/innovus_pnr.tcl

# OR: Launch interactive GUI, then source scripts step by step
innovus
  > source scripts/step1_init.tcl
  > source scripts/step2_floorplan.tcl
  > source scripts/step3_power.tcl
  > source scripts/step4_place.tcl
  > source scripts/step5_cts.tcl
  > source scripts/step6_route.tcl
  > source scripts/step7_export.tcl
```

Step 1: Initialization

scripts/step1_init.tcl

```
catch {freeDesign}

# Read MMMC configuration
read_mmmc scripts/mmmc.tcl

# Read physical LEF files
read_physical -lefs {
    $GPDK/gsclib045_tech/lef/gsclib045_tech.lef
    $GPDK/gsclib045/lef/gsclib045_macro.lef
}

# Read synthesized gate netlist
read_netlist netlist/and_gate_synth.v -top and_gate

# Initialize the design database
init_design
puts "=== Step 1: Init complete ==="
```

Step 2: Floorplan & Pin Placement

scripts/step2_floorplan.tcl

```
# Core: Aspect ratio 1.0 (square), utilization 65%
# Margins: Left=10um Bottom=10um Right=10um Top=10um
floorPlan -r 1.0 0.65 10 10 10 10

# Place ports on Metal3 on appropriate sides
editPin -side West -pin {a b} -layer Metal3
```

```
editPin -side East -pin {y}   -layer Metal3

puts "=== Step 2: Floorplan complete ==="
```

Step 3: Power Planning (PDN)

scripts/step3_power.tcl

```
# Connect power/ground nets to all cell pins
globalNetConnect VDD -type pgpin -pin VDD -inst * -all
globalNetConnect VSS -type pgpin -pin VSS -inst * -all

# Core power rings (Metal4 horizontal, Metal5 vertical)
addRing -type core_rings -nets {VDD VSS} \
    -layer {top Metal5 bottom Metal5 left Metal4 right Metal4} \
    -width 1.5 -spacing 0.5 -offset 2.0

# Vertical VDD/VSS stripes on Metal5
addStripe -nets {VDD VSS} -layer Metal5 -direction vertical \
    -width 1.0 -spacing 0.5 -set_to_set_distance 40

# Connect standard cell rails (Metal1 follow-pins)
sroute -connect {blockPin padPin corePin} \
    -layerChangeRange {Metal1 Metal5} \
    -nets {VDD VSS}

puts "=== Step 3: Power planning complete ==="
```

Step 4: Placement

scripts/step4_place.tcl

```
# Insert boundary end-cap cells
catch {addEndCap -prefix ENDCAP}

# Insert well-tap cells to prevent latch-up (every 30 um)
catch {addWellTap -cell WELLTAP -cellInterval 30 -prefix WELLTAP}

# Timing-driven standard cell placement
place_opt_design

# Verify placement legality (no overlaps)
checkPlace

# Pre-CTS timing report (ideal clocks)
timeDesign -preCTS -outDir reports/preCTS

puts "=== Step 4: Placement complete ==="
```

Step 5: CTS (Clock Tree Synthesis)

scripts/step5_cts.tcl

```
# Note: AND gate is combinational (no clock).
# CTS step is included here for completeness and
# is required in sequential designs.
# For combinational-only: run a timing check
```

```
timeDesign -preCTS -outDir reports/preCTS

# ■■■ For sequential designs: ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# set_ccopt_property buffer_cells {CLKBUFX2 CLKBUFX4 BUFX4}
# set_ccopt_property inverter_cells {INVX1 INVX2 CLKINVX1}
# set_ccopt_property target_skew 0.1
# set_ccopt_property target_max_trans 0.3
# create_ccopt_clock_tree_spec -file ccopt.spec
# source ccopt.spec
# ccopt_design -cts
# timeDesign -postCTS -outDir reports/postCTS
# optDesign  -postCTS
# optDesign  -postCTS -hold

puts "=== Step 5: CTS complete ==="
```

Step 6: Filler Insertion & Routing

■ scripts/step6_route.tcl

```
# Insert filler cells to fill all standard-cell row gaps
catch {addFiller -cell {FILL64 FILL32 FILL16 FILL8 FILL4 FILL2 FILL1} \
                 -prefix FILLER}

# Verify no gaps remain
checkPlace

# Run NanoRoute detail routing
routeDesign

# Enable OCV analysis with CPPR
setAnalysisMode -analysisType onChipVariation -cppr both

# Post-route timing optimization (setup + hold)
optDesign -postRoute
optDesign -postRoute -hold

# Post-route timing verification
timeDesign -postRoute      -outDir reports/postRoute
timeDesign -postRoute -hold -outDir reports/postRoute_hold

puts "=== Step 6: Routing complete ==="
```

STEP 12 — RC Extraction & Timing Signoff

After routing, parasitics (resistance R and capacitance C) of all metal interconnects are extracted into **SPEF** format and back-annotated into the static timing analysis engine for accurate signoff-quality timing verification.

■ scripts/step7_export.tcl

```
################################################################
# File: scripts/step7_export.tcl
# Step 7: Physical Verification, RC Extraction, GDSII Export
################################################################

# ■■ Quick DRC & connectivity check (inside Innovus) ■■■■■■■■■■
verify_drc
verifyConnectivity -type all -error 1000 -warning 50
catch {verifyProcessAntenna}

# ■■ Export post-route netlists ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Logical netlist (for LEC and gate-level simulation)
saveNetlist netlist/and_gate_postroute.v

# With power/ground pins (required for PVS LVS)
saveNetlist -includePowerGround netlist/and_gate_postroute_pg.v

# ■■ Parasitic extraction ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
setExtractRCMode -engine postRoute -effortLevel signoff
extractRC
rcOut -spef netlist/and_gate_postroute.spef

# ■■ SDF for post-route gate-level simulation ■■■■■■■■■■■■■■■■■■
write_sdf -version 3.0 netlist/and_gate_postroute.sdf

# ■■ Final signoff static timing analysis ■■■■■■■■■■■■■■■■■■■■■
timeDesign -signoff      -outDir reports/signoff
timeDesign -signoff -hold -outDir reports/signoff_hold

# ■■ GDSII stream out ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
streamOut output/and_gate.gds \
    -mapFile   $GPDK/gsclib045_tech/gds/gds2.map \
    -merge     $GPDK/gsclib045/gds/gsclib045.gds \
    -libName   and_gate \
    -structureName and_gate \
    -units 2000 \
    -mode ALL

# ■■ Save Innovus session database ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
saveDesign output/and_gate_final.enc

puts "=== PnR + Export Complete ==="
```

Key Innovus Signoff Commands Reference

Command	Description
`verify_drc`	Internal DRC check (quick, pre-PVS)
`verifyConnectivity -type all`	Check power/ground/signal connectivity

`verifyProcessAntenna`	Check metal antenna rule violations
`saveNetlist <file>`	Export logical post-route netlist
`saveNetlist -includePowerGround`	Export netlist with VDD/VSS pins (LVS)
`setExtractRCMode -engine postRoute`	Configure RC extractor engine
`extractRC`	Run parasitic RC extraction
`rcOut -spef <file>`	Write SPEF file (parasitics)
`write_sdf -version 3.0 <file>`	Write SDF timing annotation file
`timeDesign -signoff`	Final STA with extracted parasitics
`streamOut <gds> -mapFile ...`	Stream GDSII output with layer map
`saveDesign <file>`	Save Innovus database for re-entry

STEP 13–14 — Physical Verification: DRC & LVS (Cadence PVS)

Cadence **PVS** (Physical Verification System) is the signoff physical verification tool. **DRC** checks that the GDSII layout conforms to all foundry manufacturing rules. **LVS** proves that the layout connectivity exactly matches the logical gate netlist.

DRC — Design Rule Check

scripts/run_pvs_drc.csh

```
#!/bin/csh -f
# File: scripts/run_pvs_drc.csh
# Cadence PVS DRC Signoff — and_gate

mkdir -p pvs_output/drc

pvs -drc \
    -gds      output/and_gate.gds \
    -top_cell and_gate \
    -run_dir  pvs_output/drc \
    /home/ec/install/FOUNDRY/analog/45nm/pvs/pvlDRC.rul

echo "DRC Report: pvs_output/drc/and_gate.sum"
```

✔ *Signoff requires:* ***Total DRC Violations: 0*** *in pvs_output/drc/and_gate.sum*

LVS — Layout Versus Schematic

scripts/run_pvs_lvs.csh

```
#!/bin/csh -f
# File: scripts/run_pvs_lvs.csh
# Cadence PVS LVS Signoff — and_gate

mkdir -p pvs_output/lvs

set GPDK = /home/ec/install/FOUNDRY/lan/flow/t1u1/reference_libs

pvs -lvs \
    -automatch \
    -hcell            scripts/hcells.txt \
    -gds              output/and_gate.gds \
    -top_cell         and_gate \
    -source_top_cell  and_gate \
    -source_verilog   netlist/and_gate_postroute_pg.v \
    -source_cdl $GPDK/GPDK045/gsclib045_svt_v4.4/gsclib045/cdl/gsclib045.cdl \
    -control          scripts/pvs_lvs.ctrl \
    -run_dir          pvs_output/lvs \
    /home/ec/install/FOUNDRY/analog/45nm/pvs/pvlLVS.rul

echo "LVS Report: pvs_output/lvs/and_gate.rep"
```

✔ *Signoff requires:* ***LVS STATUS: CORRECT — LAYOUT AND SOURCE NETLIST ARE EQUIVALENT***

STEP 15 — GDSII Stream Out & Tapeout

The final step streams the physical layout to **GDSII** (Graphic Design System II) format — the standard mask-data format delivered to the silicon foundry for photomask fabrication. The **layer map file** translates Innovus internal layer names (Metal1, Metal2…) to GDS layer/datatype numbers required by the PDK.

Output Deliverables Summary

File	Path	Purpose
Gate Netlist	netlist/and_gate_synth.v	Synthesis output (Genus)
Synthesized SDC	netlist/and_gate_synth.sdc	Timing constraints for PnR
Post-Route Netlist	netlist/and_gate_postroute.v	For LEC and gate-sim
PG Netlist	netlist/and_gate_postroute_pg.v	For PVS LVS (has VDD/VSS)
SPEF	netlist/and_gate_postroute.spef	Parasitic RC extraction
SDF	netlist/and_gate_postroute.sdf	Timing annotation for sim
GDSII	output/and_gate.gds	Final tapeout mask data
Innovus DB	output/and_gate_final.enc	Re-entry / ECO database
DRC Report	pvs_output/drc/and_gate.sum	DRC signoff results
LVS Report	pvs_output/lvs/and_gate.rep	LVS signoff results

Master Command Cheat Sheet

Phase	Tool	Command
RTL Simulation	Cadence Xcelium	`xrun -f scripts/sim_rtl.f`
Gate-Level Sim	Cadence Xcelium	`xrun -f scripts/gate_sim.f`
Post-Route Sim	Cadence Xcelium	`xrun -f scripts/postroute_sim.f`
Logic Synthesis	Cadence Genus	`genus -f scripts/genus_synth.tcl`
Formal Verification	Conformal LEC	`lec -xl -nogui -do scripts/conformal_lec.tcl`
Place & Route (batch)	Cadence Innovus	`innovus -stylus -f scripts/innovus_pnr.tcl`
Interactive PnR	Cadence Innovus	`innovus`
DRC Signoff	Cadence PVS	`csh scripts/run_pvs_drc.csh`
LVS Signoff	Cadence PVS	`csh scripts/run_pvs_lvs.csh`

Common Debug Quick-Fixes

Issue	Symptom	Fix
Negative setup slack	timeDesign shows WNS < 0	`optDesign -postRoute -setup` `setOptMode -effort high`
Hold violations	WHS < 0 after routing	`deleteFiller` → `optDesign -postRoute -hold` → `ecoRoute` → `addFiller`
DRC errors	pvs_output/drc/*.sum non-zero	`Check antenna, min-width, spacing;` `rerun routeDesign -fixDRC`
LVS mismatch	PG netlist mismatch	`Use saveNetlist -includePowerGround;` `check hcells.txt CDL mapping`
Elaboration fail	check_design: unresolved refs	`Check .init_hdl_search_path;` `confirm module name matches file name`